



RNN Introduction

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

Sequence Data এবং Standard RNN — সম্পূর্ণ বাংলা নোট

বিষয়: Introduction to Sequence Data and Standard RNN — Why CNN/ANN cannot handle sequence data, Concept of Hidden State, and Unrolling RNN

পর্ব: ৪ — টেক্সট এবং সিকোয়েন্স ডেটা (RNN)

তারিখ: ২০২৬-০৪-১১

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

ধরো তুমি একটি গল্পের বই পড়ছ। প্রতিটি বাক্য বোঝার জন্য তোমাকে আগের বাক্যটির কথা মনে রাখতে হয়। যদি তুমি প্রতিটি বাক্য আলাদাভাবে পড়ো এবং আগের কিছু মনে না রাখো, তাহলে গল্পের কিছুই বুঝবে না।

Sequence Data হলো এমন ডেটা যেখানে **ক্রম (order)** গুরুত্বপূর্ণ। যেমন:

- বাংলা বাক্য: "আমি ভাত খাই" — এখানে শব্দের ক্রম বদলালে অর্থ পাল্টে যায়
- শেয়ার বাজারের দাম: আজকের দাম গতকালের উপর নির্ভর করে
- আবহাওয়ার পূর্বাভাস: আজকের তাপমাত্রা গতকালের উপর নির্ভরশীল
- কথা বলার audio: প্রতিটি শব্দ আগের শব্দের সাথে সম্পর্কিত

বাস্তব জীবনের উদাহরণ (রান্নার রেসিপি দিয়ে বোঝাই):

কল্পনা করো একজন রাঁধুনি রান্না করছে। সে যদি প্রতিটি পদক্ষেপ আলাদাভাবে দেখে কিন্তু আগে কী করেছিল মনে না থাকে, তাহলে রান্না নষ্ট হবে। সে জানতে পারবে না যে লবণ আগেই দেওয়া হয়েছে কিনা!

ধাপ ১: তেল গরম করো	← মনে রাখো
ধাপ ২: পেঁয়াজ ভাজো	← মনে রাখো (তেল ইতিমধ্যে গরম)
ধাপ ৩: মসলা দাও	← মনে রাখো (পেঁয়াজ ইতিমধ্যে ভাজা)
ধাপ ৪: মাংস দাও	← মনে রাখো (মসলা ইতিমধ্যে দেওয়া)
ধাপ ৫: লবণ দাও	← আগের সব ধাপ মাথায় রেখে সিদ্ধান্ত নাও

ANN/CNN এর মতো নেটওয়ার্কগুলো প্রতিটি ধাপ আলাদাভাবে দেখে — তারা আগের ধাপ মনে রাখতে পারে না। **RNN** হলো সেই রাঁধুনি যে প্রতিটি ধাপে আগের সব মনে রেখে কাজ করে!

কোন সমস্যা সমাধান করে?

RNN এই তিনটি মূল সমস্যা সমাধান করে:

1. **Variable Length Input** — বিভিন্ন দৈর্ঘ্যের sequence নিতে পারে
2. **Sequential Dependency** — আগের তথ্য মনে রেখে পরবর্তী সিদ্ধান্ত নেয়
3. **Parameter Sharing** — একই weight বারবার ব্যবহার করে, মেমোরি সাশ্রয় হয়

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ Sequence Data কী?

Sequence Data মানে এমন তথ্য যেখানে প্রতিটি element-এর একটি নির্দিষ্ট অবস্থান (position) আছে এবং সেই অবস্থান পরিবর্তন করলে অর্থ পাল্টে যায়।

উদাহরণ:

- **Text/NLP:** "কুকুর মানুষকে কামড়ায়" ≠ "মানুষ কুকুরকে কামড়ায়"
- **Time Series:** সোমবারের শেয়ার দাম → মঙ্গলবার → বুধবার (ক্রম গুরুত্বপূর্ণ)
- **Audio:** শব্দ তরঙ্গের প্রতিটি sample একটি sequence
- **Video:** প্রতিটি frame একটি sequence

২.২ কেন ANN (Artificial Neural Network) Sequence Data handle করতে পারে না?

ANN এর দুটি মৌলিক সমস্যা আছে:

সমস্যা ১: Fixed Input Size (নির্দিষ্ট আকারের input)

ANN-এর input layer-এ নির্দিষ্ট সংখ্যক neuron থাকে। তুমি যদি 100টি neuron দিয়ে ANN বানাও, তাহলে সে সবসময় 100টি feature নেবে। কিন্তু:

- "আমি ভাত খাই" = ৩টি শব্দ
- "আমি প্রতিদিন সকালে বাড়িতে ভাত খাই" = ৭টি শব্দ

ANN এই দুটি বাক্য একসাথে handle করতে পারবে না কারণ তার input size fixed!

ANN এর দৃষ্টিভঙ্গি:

[শব্দ_১, শব্দ_২, শব্দ_৩, ০, ০, ০, ০] ← padding করতে হয়

[শব্দ_১, শব্দ_২, শব্দ_৩, শব্দ_৪, শব্দ_৫, শব্দ_৬, শব্দ_৭]

সমস্যা ২: No Memory (কোনো স্মৃতি নেই)

ANN প্রতিটি input কে সম্পূর্ণ স্বাধীনভাবে দেখে। সে বুঝতে পারে না যে আগের input-এর সাথে বর্তমান input-এর কী সম্পর্ক।

ANN এর চিন্তাভাবনা:

Input: "আমি" → Output: ??? (বাকি শব্দ জানে না)

Input: "ভাত" → Output: ??? (আগের শব্দ মনে নেই)

Input: "খাই" → Output: ??? (আগের দুটো শব্দ মনে নেই)

সমস্যা ৩: Word Position Ignored (শব্দের অবস্থান উপেক্ষা)

ANN একটি বাক্যের সব শব্দকে একসাথে দেখে, কিন্তু কোন শব্দ আগে এবং কোন শব্দ পরে সেটা সে বোঝে না।

২.৩ কেন CNN (Convolutional Neural Network) Sequence Data handle করতে পারে না?

CNN মূলত **spatial patterns** (স্থানিক নকশা) খুঁজে বের করার জন্য তৈরি। Image-এ একটি চোখের pattern যেকোনো জায়গায় থাকতে পারে — CNN সেটা ধরতে পারে।

CNN এর সমস্যা:

1. **Local Features Only:** CNN শুধু কাছাকাছি elements-এর মধ্যে pattern খোঁজে। "আমি আজ... খুশি" — এখানে "আমি" এবং "খুশি" অনেক দূরে থাকলে CNN এই long-range dependency ধরতে পারে না।
2. **No Temporal Memory:** CNN-এ সময়ের concept নেই। সে জানে না কোনটা আগে এবং কোনটা পরে এসেছে।
3. **Fixed Window:** CNN একটি নির্দিষ্ট আকারের window দিয়ে দেখে, তাই variable-length sequence handle করতে পারে না।

CNN এর দেখার পদ্ধতি (Image-এর জন্য ভালো):

[পিক্সেল] [পিক্সেল] [পিক্সেল]

[পিক্সেল] [পিক্সেল] [পিক্সেল] ← 2D spatial pattern

[পিক্সেল] [পিক্সেল] [পিক্সেল]

Text-এর জন্য সমস্যা:

"আমি" → "ভাত" → "খাই" ← এখানে time dimension আছে, CNN বোঝে না

২.৪ RNN কীভাবে কাজ করে?

RNN (Recurrent Neural Network) একটি বিশেষ ধরনের neural network যেখানে **loop** আছে। এই loop-এর মাধ্যমে, network আগের step-এর তথ্য পরের step-এ পাঠায়।

RNN-এর মূল বৈশিষ্ট্য:

- প্রতিটি time step-এ একটি input নেয়
- আগের step থেকে একটি **hidden state** পায়
- নতুন hidden state তৈরি করে
- একটি output দেয়

২.৫ Hidden State (লুকানো অবস্থা) কী?

Hidden State হলো RNN-এর "স্মৃতি" বা "মেমোরি"। এটি একটি vector যেটিতে আগের সব time step-এর সারসংক্ষেপ থাকে।

সহজ ভাষায় বললে:

h_t = "এখন পর্যন্ত যা দেখেছি তার সারসংক্ষেপ"

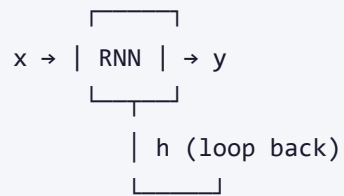
Hidden state কীভাবে কাজ করে:

- প্রথম step-এ: h_0 = শূন্য vector (কিছুই জানে না)
- দ্বিতীয় step-এ: h_1 = প্রথম শব্দ থেকে শেখা জ্ঞান
- তৃতীয় step-এ: h_2 = প্রথম + দ্বিতীয় শব্দ থেকে শেখা জ্ঞান
- এভাবে চলতে থাকে...

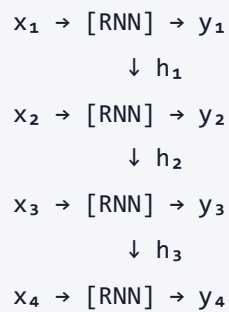
২.৬ Unrolling (প্রসারিত করা) কী?

RNN-এ একটি loop থাকে যেটি বারবার চলে। **Unrolling** মানে এই loop-টিকে সময়ের সাথে সাথে বিছিয়ে দেওয়া — প্রতিটি time step-কে আলাদা করে দেখানো।

Rolled (গোড়ানো) RNN:



Unrolled (প্রসারিত) RNN:



৩. Math / Theory

৩.১ RNN-এর মূল সমীকরণ

RNN-এর দুটি মূল equation আছে:

Equation 1: Hidden State Update

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

Equation 2: Output

$$y_t = W_y \cdot h_t + b_y$$

৩.২ প্রতিটি Symbol-এর অর্থ

Symbol	বাংলায় অর্থ	আকার (Shape)
x_t	t-তম time step-এর input	(input_size,)
h_t	t-তম time step-এর hidden state	(hidden_size,)
h_{t-1}	আগের time step-এর hidden state	(hidden_size,)
y_t	t-তম time step-এর output	(output_size,)
W_h	hidden-to-hidden weight matrix	(hidden_size, hidden_size)
W_x	input-to-hidden weight matrix	(hidden_size, input_size)
W_y	hidden-to-output weight matrix	(output_size, hidden_size)
b_h	hidden state-এর bias	(hidden_size,)
b_y	output-এর bias	(output_size,)
$\tanh$	Hyperbolic Tangent activation function	—

৩.৩ Manual Calculation (সংখ্যা দিয়ে হিসাব)

ধরো একটি ছোট উদাহরণ:

- Input size = 2 (প্রতিটি শব্দ 2-dimensional vector)
- Hidden size = 3
- একটি বাক্য: $[x_1, x_2] = [[1, 0], [0, 1]]$

Step 1: Initial Hidden State

$$h_0 = [0, 0, 0] \leftarrow \text{শুরুতে শূন্য}$$

Step 2: Weight Matrices (ধরে নিচ্ছি)

$$\begin{aligned} W_x &= \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix}, & W_h &= \begin{bmatrix} 0.3 & 0.1 & 0.2 \\ 0.2 & 0.4 & 0.1 \\ 0.1 & 0.2 & 0.3 \end{bmatrix}, & b_h &= [0.1, 0.1, 0.1] \end{aligned}$$

Step 3: First Time Step ($x_1 = [1, 0]$)

```

raw = W_x · x1 + W_h · h0 + b_h
    = [0.1×1 + 0.2×0,      = [0.1,      + [0, 0, 0] + [0.1, 0.1, 0.1]
      0.3×1 + 0.4×0,      0.3,
      0.5×1 + 0.6×0]      0.5]

raw = [0.1+0+0.1,  0.3+0+0.1,  0.5+0+0.1]
raw = [0.2, 0.4, 0.6]

h1 = tanh([0.2, 0.4, 0.6])
h1 ≈ [0.197, 0.380, 0.537] ← এটি প্রথম শব্দের "স্মৃতি"

```

Step 4: Second Time Step ($x_2 = [0, 1]$)

```

raw = W_x · x2 + W_h · h1 + b_h

W_x · x2 = [0.2, 0.4, 0.6]

W_h · h1 = [0.3×0.197 + 0.1×0.380 + 0.2×0.537,
            0.2×0.197 + 0.4×0.380 + 0.1×0.537,
            0.1×0.197 + 0.2×0.380 + 0.3×0.537]
          ≈ [0.059+0.038+0.107, 0.039+0.152+0.054, 0.020+0.076+0.161]
          ≈ [0.204, 0.245, 0.257]

raw = [0.2+0.204+0.1, 0.4+0.245+0.1, 0.6+0.257+0.1]
raw = [0.504, 0.745, 0.957]

h2 = tanh([0.504, 0.745, 0.957])
h2 ≈ [0.468, 0.631, 0.744] ← এটি দুটো শব্দের মিলিত "স্মৃতি"

```

লক্ষ্য করো: h_2 তে x_1 এবং x_2 উভয়ের তথ্য আছে! এটাই RNN-এর জাদু।

৩.৪ tanh Activation Function

tanh function আউটপুটকে -1 থেকে +1 এর মধ্যে সীমাবদ্ধ রাখে:

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

$$\tanh(0.0) = 0.000$$

$$\tanh(0.5) = 0.462$$

$$\tanh(1.0) = 0.762$$

$$\tanh(2.0) = 0.964$$

$$\tanh(-1.0) = -0.762$$

tanh কেন ব্যবহার করা হয়:

- Hidden state-এর মান -1 থেকে +1 এর মধ্যে থাকে (explode করে না)
- Zero-centered (0 এর কাছাকাছি থাকে) তাই gradient হিসাব ভালো হয়

৩.৫ Backpropagation Through Time (BPTT)

RNN-এ learning হয় **BPTT** এর মাধ্যমে। Unrolled RNN-এর প্রতিটি time step-এ error পিছনে (backward) পাঠানো হয়।

Forward: $x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow x_3 \rightarrow h_3 \rightarrow \text{Loss}$

Backward: $\text{Loss} \leftarrow h_3 \leftarrow h_2 \leftarrow h_1$ (gradient পিছনে যায়)

8. Code Example (Python)

8.১ NumPy দিয়ে RNN from scratch


```

import numpy as np

# =====
# RNN from scratch – NumPy দিয়ে তৈরি
# =====

# Seed নির্ধারণ (reproducibility এর জন্য)
np.random.seed(42)

# — Hyperparameters —————
input_size = 4    # প্রতিটি input vector-এর আকার
hidden_size = 3    # hidden state-এর আকার
output_size = 2    # output-এর আকার
seq_length = 5    # sequence-এর দৈর্ঘ্য

# — Weight Initialization —————
# input থেকে hidden-এ যাওয়ার weight
Wx = np.random.randn(hidden_size, input_size) * 0.01

# hidden থেকে hidden-এ যাওয়ার weight (recurrent connection)
Wh = np.random.randn(hidden_size, hidden_size) * 0.01

# hidden থেকে output-এ যাওয়ার weight
Wy = np.random.randn(output_size, hidden_size) * 0.01

# Bias terms
bh = np.zeros((hidden_size, 1))    # hidden state bias
by = np.zeros((output_size, 1))    # output bias

print("=" * 50)
print("RNN Weight Shapes:")
print(f" Wx (input→hidden): {Wx.shape}")
print(f" Wh (hidden→hidden): {Wh.shape}")
print(f" Wy (hidden→output): {Wy.shape}")
print("=" * 50)

# — RNN Forward Pass (এগিয়ে যাওয়া) —————
def rnn_forward(inputs, h_prev):
    """
    RNN-এর forward pass।
    inputs: list of input vectors (প্রতিটি shape: input_size x 1)

```

```

h_prev: initial hidden state (shape: hidden_size x 1)
"""

# সব hidden states এবং outputs সংরক্ষণ করো
hidden_states = {}
outputs = []

# শুরুতে h_prev সংরক্ষণ করো (t=-1 এর মতো)
hidden_states[-1] = h_prev.copy()

# প্রতিটি time step-এ কাজ করো
for t, x_t in enumerate(inputs):

    # Step 1: Input reshape (column vector বানাও)
    x_t = x_t.reshape(-1, 1) # (input_size, 1) আকার করো

    # Step 2: Hidden state update (মূল RNN সমীকরণ)
    #  $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$ 
    raw = np.dot(W_h, hidden_states[t-1]) + np.dot(W_x, x_t) + b_h
    h_t = np.tanh(raw) # activation function লাগাও

    # Step 3: Output তৈরি করো
    #  $y_t = W_y \cdot h_t + b_y$ 
    y_t = np.dot(W_y, h_t) + b_y

    # Step 4: সংরক্ষণ করো
    hidden_states[t] = h_t
    outputs.append(y_t)

    # প্রতিটি step-এর অবস্থা দেখাও
    print(f"\n Time Step t={t+1}:")
    print(f"    Input x_{t+1}:          {x_t.flatten()}")
    print(f"    Hidden State h_{t+1}:    {h_t.flatten().round(4)}")
    print(f"    Output y_{t+1}:          {y_t.flatten().round(4)}")

return outputs, hidden_states

# — একটি উদাহরণ sequence তৈরি করো —————
print("\n 🐼 একটি 5-শব্দের বাক্য process করছি...")
print("(প্রতিটি শব্দ 4-dimensional vector দিয়ে represent করা হয়েছে)")
print()

# ৫টি শব্দের sequence (প্রতিটি 4-dimensional)

```

```
input_sequence = [  
    np.array([1, 0, 0, 0]), # শব্দ ১: "আমি"  
    np.array([0, 1, 0, 0]), # শব্দ ২: "ভাত"  
    np.array([0, 0, 1, 0]), # শব্দ ৩: "খুব"  
    np.array([0, 0, 0, 1]), # শব্দ ৪: "ভালো"  
    np.array([1, 1, 0, 0]), # শব্দ ৫: "খাই"  
]  
  
# Initial hidden state (শুরুতে শূন্য)  
h0 = np.zeros((hidden_size, 1))  
  
print(f" Initial Hidden State h_0: {h0.flatten()}")  
print()  
  
# Forward pass চালাও  
outputs, hidden_states = rnn_forward(input_sequence, h0)  
  
print("\n" + "=" * 50)  
print("✅ Forward Pass সম্পন্ন!")  
print(f" মোট {len(outputs)} টি output তৈরি হয়েছে")  
print(f" শেষ hidden state: {hidden_states[4].flatten().round(4)}")  
print("=" * 50)
```

Expected Output:

```
=====
RNN Weight Shapes:
Wx (input→hidden): (3, 4)
Wh (hidden→hidden): (3, 3)
Wy (hidden→output): (2, 3)
=====
```

🖼️ একটি 5-শব্দের বাক্য process করছি...
(প্রতিটি শব্দ 4-dimensional vector দিয়ে represent করা হয়েছে)

Initial Hidden State h_0 : [0. 0. 0.]

Time Step $t=1$:

Input x_1 : [1 0 0 0]
Hidden State h_1 : [0.0050 -0.0098 0.0062]
Output y_1 : [-0.0001 0.0001]

Time Step $t=2$:

Input x_2 : [0 1 0 0]
Hidden State h_2 : [-0.0035 -0.0150 0.0083]
...

✅ Forward Pass সম্পন্ন!

8.২ TensorFlow/Keras দিয়ে RNN

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# =====
# TensorFlow/Keras দিয়ে সহজে RNN তৈরি
# =====

# — Sentiment Analysis: কোনো বাক্য positive না negative —
# বাংলা উদাহরণ ব্যবহার করা হয়েছে (English শব্দ)

# Training Data (ছোট উদাহরণের জন্য)
texts = [
    "i love this movie",      # positive (1)
    "this is amazing",       # positive (1)
    "great film loved it",   # positive (1)
    "wonderful experience",  # positive (1)
    "i hate this movie",     # negative (0)
    "terrible film awful",   # negative (0)
    "worst experience ever", # negative (0)
    "boring and dull",       # negative (0)
]

labels = [1, 1, 1, 1, 0, 0, 0, 0] # 1=positive, 0=negative

# — Tokenization (শব্দকে সংখ্যায় রূপান্তর) —
tokenizer = Tokenizer(num_words=50) # সর্বোচ্চ 50 ধরনের শব্দ
tokenizer.fit_on_texts(texts)       # শব্দের অভিধান তৈরি
sequences = tokenizer.texts_to_sequences(texts) # শব্দ → সংখ্যা

print("শব্দের অভিধান (Word Index):")
print(tokenizer.word_index)         # প্রতিটি শব্দের নম্বর
print()

# — Padding (সব sequence একই দৈর্ঘ্যের করো) —
max_length = 5                     # সর্বোচ্চ ৫টি শব্দ
X = pad_sequences(sequences, maxlen=max_length, padding='post')
y = np.array(labels)

print("Padded Sequences (সংখ্যায় রূপান্তরিত বাক্য):")

```

```

for i, (text, seq) in enumerate(zip(texts, X)):
    print(f" '{text}' → {seq}")
print()

# — RNN Model তৈরি —————
vocab_size = 50      # মোট শব্দের সংখ্যা
embed_dim  = 8       # প্রতিটি শব্দের embedding dimension

model = Sequential([
    # Layer 1: Embedding – শব্দকে vector-এ রূপান্তর
    Embedding(input_dim=vocab_size,  # শব্দের পরিধি
              output_dim=embed_dim,  # vector আকার
              input_length=max_length,
              name='embedding'),

    # Layer 2: SimpleRNN – মূল recurrent layer
    SimpleRNN(units=16,              # hidden state-এর আকার
              activation='tanh',      # tanh activation
              return_sequences=False, # শুধু শেষ output দাও
              name='rnn_layer'),

    # Layer 3: Dense output – Binary classification
    Dense(units=1,                   # একটি output (0 বা 1)
          activation='sigmoid',      # sigmoid (0-1 এর মধ্যে)
          name='output_layer'),
])

# Model compile করো
model.compile(
    optimizer='adam',              # Adam optimizer
    loss='binary_crossentropy',    # binary classification loss
    metrics=['accuracy']           # accuracy track করো
)

# Model এর structure দেখো
print("=" * 55)
print("Model Architecture:")
print("=" * 55)
model.summary()
print()

# — Model Train করো —————
print("Training শুরু হচ্ছে...")

```

```
history = model.fit(
    X, y,
    epochs=50,          # ৫০ বার data দিয়ে train করো
    batch_size=4,       # একসাথে ৪টি example দিয়ে শেখো
    validation_split=0.2, # ২০% validation data রাখো
    verbose=0           # output কম দেখাও
)

# শেষ epoch-এর accuracy
final_acc = history.history['accuracy'][-1]
print(f"Final Training Accuracy: {final_acc:.2%}")
print()

# — Test করো —————
test_texts = [
    "i love this",
    "terrible movie",
]

test_sequences = tokenizer.texts_to_sequences(test_texts)
test_padded = pad_sequences(test_sequences, maxlen=max_length, padding='post')

predictions = model.predict(test_padded, verbose=0)

print("Prediction Results:")
for text, pred in zip(test_texts, predictions):
    sentiment = "✅ Positive" if pred[0] > 0.5 else "❌ Negative"
    print(f" '{text}' → {sentiment} (score: {pred[0]:.3f})")
```

8.৩ RNN দিয়ে Time Series Prediction

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
import matplotlib
matplotlib.use('Agg') # GUI ছাড়া চালানোর জন্য

# =====
# RNN দিয়ে Sine Wave Prediction (Time Series)
# =====

# — Sine Wave Data তৈরি করে —————
t = np.linspace(0, 4 * np.pi, 200) # 0 থেকে 4π পর্যন্ত
data = np.sin(t) # sine function

print(f"Data Shape: {data.shape}") # (200,)
print(f"Data Range: [{data.min():.2f}, {data.max():.2f}]")

# — Sequence তৈরি করে —————
# প্রতি 20টি মান দেখে পরের মান predict করে
timesteps = 20 # কতগুলো আগের মান দেখবে

def create_sequences(data, timesteps):
    """
    Sliding window দিয়ে sequences তৈরি করে।
    X: আগের timesteps টি মান
    y: পরের একটি মান (predict করতে হবে)
    """
    X, y = [], []
    for i in range(len(data) - timesteps):
        X.append(data[i : i + timesteps]) # আগের 20টি মান
        y.append(data[i + timesteps]) # পরের ১টি মান
    return np.array(X), np.array(y)

X, y = create_sequences(data, timesteps)

# RNN এর জন্য shape adjust করে: (samples, timesteps, features)
X = X.reshape(X.shape[0], X.shape[1], 1)

# Train/Test split
split = int(len(X) * 0.8) # ৮০% train, ২০% test
X_train, X_test = X[:split], X[split:]

```



```
y_train, y_test = y[:split], y[split:]

print(f"\nTraining Samples: {X_train.shape}")
print(f"Test Samples: {X_test.shape}")

# — RNN Model —————
model = Sequential([
    # SimpleRNN layer: ২০টি পূর্ববর্তী মান থেকে pattern শেখে
    SimpleRNN(units=32,
               activation='tanh',
               input_shape=(timesteps, 1), # (20 time steps, 1 feature)
               name='rnn'),

    # Dense layer: একটি output predict করে
    Dense(1, name='output'),
])

model.compile(optimizer='adam', loss='mse')

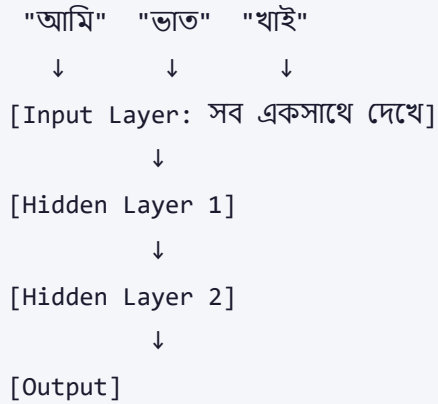
# Train
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=0
)

# Test Loss
test_loss = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Loss (MSE): {test_loss:.6f}")
print(f"Test RMSE: {np.sqrt(test_loss):.6f}")
print("✅ RNN দিয়ে Time Series Prediction সফল!")
```

৫. 🎨 Visual / Diagram

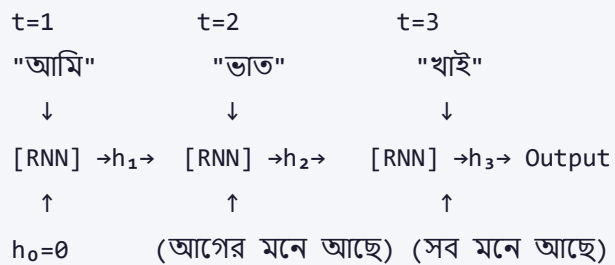
৫.১ ANN vs RNN — মূল পার্থক্য

ANN (সাধারণ নেটওয়ার্ক)



⚠️ সমস্যা: সব শব্দ একসাথে দেখে, ক্রম বোঝে না!
 "আমি ভাত খাই" = "ভাত আমি খাই" (ANN এর কাছে একই!)

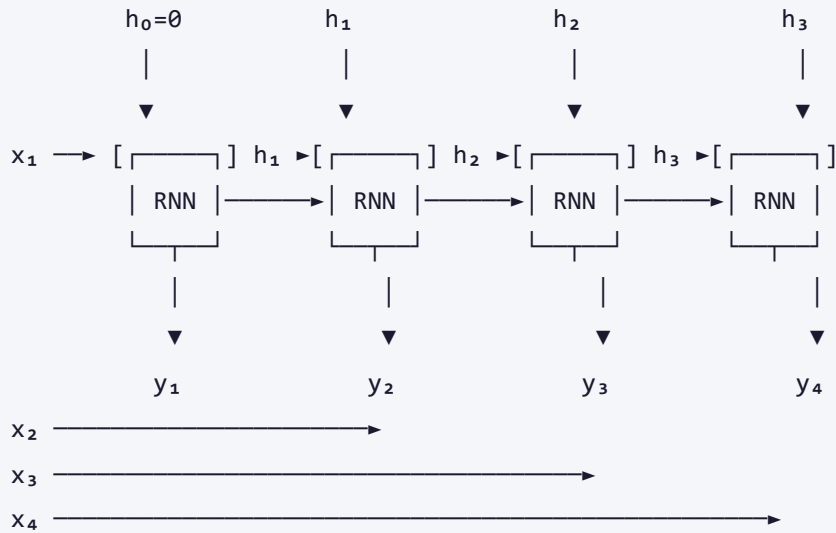
RNN (Recurrent নেটওয়ার্ক)



✅ প্রতিটি step-এ আগের তথ্য মনে রাখে!

৫.২ RNN Unrolled Diagram (বিস্তারিত)

RNN – Unrolled View



মনে রাখো: প্রতিটি RNN cell-এ একই Weight (w_x , w_h , w_y) ব্যবহার হয়!
এটাই Parameter Sharing!

৫.৩ Hidden State এর Evolution

Hidden State-এর বিবর্তন

$h_0 = [0.00, 0.00, 0.00] \leftarrow$ শুরুতে ফাঁকা (কিছু জানে না)

↓

↓ "আমি" input

$h_1 = [0.12, -0.08, 0.15] \leftarrow$ "আমি" সম্পর্কে ধারণা তৈরি

↓

↓ "ভাত" input

$h_2 = [0.45, 0.23, -0.31] \leftarrow$ "আমি" + "ভাত" উভয়ের ধারণা

↓

↓ "খাই" input

$h_3 = [0.67, 0.51, 0.44] \leftarrow$ "আমি" + "ভাত" + "খাই" সব মনে আছে

শেষ hidden state h_3 পুরো বাক্যের meaning ধারণ করে!

৫.৪ RNN এর বিভিন্ন Modes

RNN-এর ভিন্ন ভিন্ন প্রকার	
---------------------------	--

1. ONE-TO-ONE (সাধারণ ANN):
 $x \rightarrow [RNN] \rightarrow y$
(Image Classification)
2. ONE-TO-MANY (একটি input, অনেক output):
 $x \rightarrow [RNN] \rightarrow y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow y_4$
(Image Captioning: একটি ছবি থেকে বাক্য তৈরি)
3. MANY-TO-ONE (অনেক input, একটি output):
 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow [RNN] \rightarrow y$
(Sentiment Analysis: বাক্য দেখে positive/negative বলা)
4. MANY-TO-MANY (অনেক input, অনেক output - same length):
 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow [RNN] \rightarrow y_1 \rightarrow y_2 \rightarrow y_3$
(POS Tagging: প্রতিটি শব্দের ব্যাকরণগত শ্রেণী বলা)
5. MANY-TO-MANY (অনেক input, অনেক output - different length):
 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow [Encoder] \rightarrow [Decoder] \rightarrow y_1 \rightarrow y_2$
(Machine Translation: বাংলা থেকে ইংরেজি অনুবাদ)

৫.৫ CNN vs RNN — Feature Comparison

বৈশিষ্ট্য	CNN	RNN
Memory	নেই	আছে (hidden state)
Input Type	Fixed size	Variable length
Best for	Images	Sequences/Text/Time
Local Patterns	খুব ভালো	মোটামুটি ভালো
Long Range Dep.	দুর্বল	ভালো (কিন্তু সীমিত)
Parallelization	সহজ	কঠিন (sequential)
Parameter Share	filter	weights (W_x, W_h, W_y)

৬. Real-world Use Cases

১. Google Translate (ভাষা অনুবাদ)

Google Translate-এর আগের version RNN ব্যবহার করত। একটি ভাষার বাক্য পড়ে অন্য ভাষায় রূপান্তর করা sequence-to-sequence কাজ।

ইনপুট: "আমি বাংলাদেশে থাকি" (বাংলা)
RNN: প্রতিটি শব্দ process করে context তৈরি
আউটপুট: "I live in Bangladesh" (ইংরেজি)

২. Siri / Google Assistant (Speech Recognition)

কথা বলার audio একটি time series। RNN প্রতিটি মুহূর্তের audio sample দেখে শব্দ চিনতে পারে।

Audio Wave → [RNN] → "আমার তারিখ কী?"

৩. Stock Price Prediction (শেয়ার বাজার)

আগের দিনের শেয়ার দাম দেখে পরের দিনের দাম predict করে।

কোম্পানি: Goldman Sachs, JPMorgan, BlackRock
Input: গত ৩০ দিনের দাম
Output: আগামীকালের সম্ভাব্য দাম

৪. Text Generation (লেখা তৈরি করা)

ChatGPT-এর আগের যুগে RNN দিয়ে text generate করা হত।

Input: "একবার এক রাজা ছিলেন যিনি..."
RNN: পরের শব্দ predict করে
Output: "...যিনি একটি বিশাল রাজত্বের অধিপতি ছিলেন"

৫. Music Generation (সঙ্গীত তৈরি)

Musical notes-এর sequence দেখে নতুন সুর তৈরি করা।

Google Magenta Project:
Input: Bach-এর কয়েকটি musical note
RNN: সঙ্গীতের pattern শেখে
Output: নতুন classical-style সঙ্গীত

৭. ⚖️ Pros & Cons

সুবিধা ✅	অসুবিধা ❌
Variable-length sequence handle করতে পারে	Sequential processing-এর কারণে ধীর গতি
আগের context মনে রাখতে পারে	Vanishing Gradient সমস্যা — দীর্ঘ sequence-এ প্রথম দিকের তথ্য ভুলে যায়
Parameter sharing-এর কারণে efficient	Exploding Gradient সমস্যা হতে পারে
যেকোনো দৈর্ঘ্যের input নিতে পারে	Parallel processing কঠিন (GPU-তে ধীর)
NLP, Audio, Time Series-এ কার্যকর	Long-term dependency ভালোভাবে capture করতে পারে না
Interpretable hidden states	LSTM/GRU-এর তুলনায় দুর্বল
Simple architecture	Real-time application-এ latency বেশি

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Hidden State Initialize না করা

```
# ❌ ভুল: hidden state উল্লেখ না করা
rnn = SimpleRNN(32) # কাজ করবে কিন্তু initial state শূন্য ধরে

# ✅ সঠিক: initial state উল্লেখ করা
initial_state = tf.zeros((batch_size, hidden_size))
output = rnn(inputs, initial_state=initial_state)
```

ভুল ২: Input Shape ভুল করা

```
# ❌ ভুল: 2D input দেওয়া
X = np.array([[1, 2, 3, 4, 5]]) # shape: (1, 5)

# ✅ সঠিক: 3D input দেওয়া (batch, timesteps, features)
X = np.array([[[1], [2], [3], [4], [5]]]) # shape: (1, 5, 1)
```

ভুল ৩: return_sequences বোঝা না

```
# return_sequences=False (default)
# শুধু শেষ time step-এর output দেয়
rnn1 = SimpleRNN(32, return_sequences=False)

# return_sequences=True
# সব time step-এর output দেয় (stacked RNN-এর জন্য দরকার)
rnn2 = SimpleRNN(32, return_sequences=True)

# ❌ ভুল: Stacked RNN-এ প্রথম layer-এ return_sequences=False দেওয়া
model = Sequential([
    SimpleRNN(32, return_sequences=False), # ভুল! দ্বিতীয় RNN data পাবে না
    SimpleRNN(16),
])

# ✅ সঠিক: Stacked RNN
model = Sequential([
    SimpleRNN(32, return_sequences=True), # সব timesteps-এর output দাও
    SimpleRNN(16), # শুধু শেষটা নাও
])
```

ভুল ৪: Vanishing Gradient সম্পর্কে অসচেতন থাকা

Standard RNN দীর্ঘ sequence-এ (50+ শব্দ) প্রথম দিকের তথ্য ভুলে যায়। এই সমস্যার সমাধান হলো LSTM বা GRU ব্যবহার করা।

```
# ❌ দীর্ঘ sequence-এর জন্য Standard RNN
model = Sequential([SimpleRNN(64)]) # 100+ শব্দের জন্য কার্যকর নয়

# ✅ দীর্ঘ sequence-এর জন্য LSTM ব্যবহার করো
from tensorflow.keras.layers import LSTM
model = Sequential([LSTM(64)]) # অনেক বেশি কার্যকর
```

ভুল ৫: Gradient Clipping না করা

```
# ❌ ভুল: Gradient Clipping ছাড়া
model.compile(optimizer='adam', loss='mse')

# ✅ সঠিক: Gradient Clipping সহ (exploding gradient রোধে)
from tensorflow.keras.optimizers import Adam
optimizer = Adam(clipvalue=1.0) # gradient ১.০ এর মধ্যে রাখো
model.compile(optimizer=optimizer, loss='mse')
```

ভুল ৬: Padded Sequences সঠিকভাবে handle না করা

```
# Padding করার পরে mask ব্যবহার করো
model = Sequential([
    Embedding(vocab_size, embed_dim, mask_zero=True), # 0 মানে padding
    SimpleRNN(32),
    Dense(1, activation='sigmoid')
])
# mask_zero=True দিলে RNN padded zeros ignore করে
```

৯. 🔗 Related Topics

আগে কী জানা দরকার?

1. ANN (Artificial Neural Network): RNN বোঝার আগে সাধারণ neural network বোঝা আবশ্যিক

- Forward Propagation কীভাবে কাজ করে
- Weights এবং Biases

1. Backpropagation: Error কীভাবে পিছনে যায় এবং weights update হয়

- Chain Rule
- Gradient Descent

1. Activation Functions: tanh, sigmoid, ReLU

1. Word Embeddings (Word2Vec, GloVe): শব্দকে vector-এ রূপান্তর

1. Numpy এবং TensorFlow/Keras-এর basic জ্ঞান

পরে কী শেখা উচিত?

1. Vanishing Gradient Problem: কেন Standard RNN দুর্বল? (পরবর্তী নোট)

1. LSTM (Long Short-Term Memory): RNN-এর উন্নত version

- Forget Gate, Input Gate, Output Gate
- Cell State

1. GRU (Gated Recurrent Unit): LSTM-এর সহজ version

1. Bidirectional RNN: উভয় দিক থেকে sequence পড়া

1. Attention Mechanism: Transformer-এর ভিত্তি

1. Transformer Architecture: আধুনিক NLP-এর মূল (ChatGPT-এর ভিত্তি)

Learning Path:

ANN → Backpropagation → Standard RNN (এখন) → Vanishing Gradient →
LSTM/GRU → Bidirectional RNN → Attention → Transformer → BERT/GPT

So. 🧠 Memory Tricks

মনে রাখার কৌশল

Hidden State মনে রাখার টেকনিক:

"RNN-এর hidden state = রাঁধুনির মাথায় রান্নার ধাপগুলোর স্মৃতি"

— প্রতিটি নতুন ingredient যোগ হলে পুরানো স্মৃতির সাথে মিশে নতুন স্মৃতি তৈরি হয়

Unrolling মনে রাখার টেকনিক:

"Unrolling = একটি clipped film-কে flat করে বিছানোর মতো"

— Loop গোছানো থাকলে বোঝা কঠিন, বিছিয়ে দিলে সহজ

RNN vs ANN পার্থক্য:

ANN = প্রতিটি পরীক্ষায় Roll Number নম্বর দিয়ে আলাদা খাতা

RNN = একটি খাতায় প্রতিটি বিষয় লিখতে থাকো — আগেরটা দেখা যায়!

গুরুত্বপূর্ণ সমীকরণ মনে রাখো:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

$\uparrow$ $\uparrow$ $\uparrow$
 "স্মৃতি" "আগের স্মৃতি" "নতুন input"

১ লাইনে সারসংক্ষেপ

"RNN হলো সেই neural network যে loop ব্যবহার করে আগের তথ্য hidden state-এ রেখে পরের সিদ্ধান্ত নেয় — ঠিক একজন মানুষের মতো যে গল্প মনে রেখে পরের কথা বলে।"

✦ Quick Reference Card

RNN Quick Reference

মূল সমীকরণ:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

$$y_t = W_y \cdot h_t + b_y$$

কোথায় ব্যবহার করবে:

- ✓ Text (NLP tasks)
- ✓ Time Series
- ✓ Audio/Speech
- ✓ Video Analysis

কোথায় এড়িয়ে চলবে:

- ✗ Image data (CNN ব্যবহার করো)
- ✗ Tabular data (ANN ব্যবহার করো)
- ✗ দীর্ঘ sequence (LSTM/GRU ব্যবহার করো)
- ✗ Real-time, fast processing দরকার হলে (Transformer ব্যবহার)

TensorFlow-এ:

```
from tensorflow.keras.layers import SimpleRNN
rnn_layer = SimpleRNN(units=64, activation='tanh')
```

📅 তৈরির তারিখ: ২০২৬-০৪-১১ | 🤖 AI-assisted Bengali ML Notes

🔗 পরবর্তী নোট: Vanishing Gradient Problem in RNN

📚 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🤖 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes